



Relational Fraud Detection in Financial Transactions Using Graph Neural Networks.

Pooja Nemade- 50678154
Rhea- 50672353
Jay- 50681732



Problem Statement and Description

The Problem: Fraud transactions are not independent events. Traditional ML models (like Logistic Regression, MLP) treat each transaction as isolated. They fail to capture hidden relationships like shared card/device/email.

Our Insights: Fraud happens in clusters or networks and suspicious transactions are connected via shared attributes.

Relationships between transactions can contain useful signals

Our Approach:

1. Convert tabular data to a Graph structure
2. Connect transactions using shared features
3. Train GNN models (GraphSAGE, GAT)
4. Detect fraud patterns across neighbors
5. Compare with baseline MLP



Dataset

Source: IEEE-CIS Fraud Detection (Kaggle) — 590k+ transactions.

Data Prep:

- Merged train_transaction.csv and train_identity.csv on TransactionID.
- Stratified Sampling: Created a 100,000-row sample to maintain the exact 3.5% fraud rate.
- Feature Engineering: Log-transformation of TransactionAmt, handling missing values in identity features, and standard scaling.



Data Preprocessing & Pipeline

1. **Data Integration:** Merged train_transaction (payment info) and train_identity (device/network info) using TransactionID.
2. **Numerical Processing:** Applied **Log-Transformation** to TransactionAmt to normalize highly skewed payment values.
3. **Standard Scaler:** Used **StandardScaler** to bring all numerical features to a common mean and variance.
4. **Categorical Handling:** Utilized **Label Encoding** for high-cardinality features (e.g., Device Type, OS, and Email Domains).
5. **Missing Data:** Implemented an imputation strategy to handle the high percentage of NaN values typically found in identity and V-series features.



Methodology: Graph Construction

Node Definition: Each transaction is represented as a node

Edge Creation: Connections (edges) are created between nodes if they share specific "High-Signal" identity features:

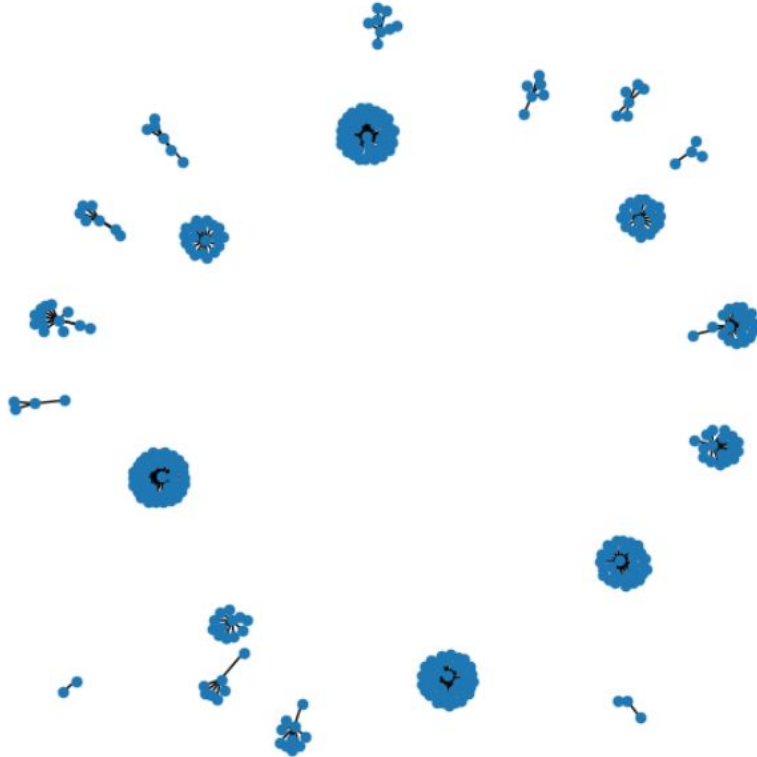
- **Financial Identifiers:** Shared credit card numbers (card1–card6) and billing addresses.
- **Digital Identity:** Shared recovery email domains (P_emaildomain) and specific device signatures (id_30 for OS, id_31 for Browser).

Graph Structure: Captures relationships between transactions that are missed in tabular data

Information Propagation: This structure allows for **Message Passing**, where a transaction "learns" from the status of its neighbors.

Detection Strategy: If a node is connected to a cluster of known fraudulent transactions, the GNN can flag it as high-risk, even if its individual features (like transaction amount) seem normal.

Graph visualization



[card1] added 912,231 edges and skipped 4677 groups

[P_emaildomain] added 3,170 edges and skipped 56 groups

[addr1] added 9,039 edges and skipped 194 groups

[card2] added 10,120 edges and skipped 488 groups

[R_emaildomain] added 7,942 edges and skipped 43 groups

Total edges: 1865684

Avg edges per node: 3.2

Model Architectures



MLP Model- Baseline model

3 Hidden Layers + 1 Output Layer

Activation: ReLU

Batch Normalization after each layer

Dropout: 0.3

This model was used to establish baseline performance.

GraphSage Model-

Uses edge_index to aggregate neighbor features

Mean aggregation from connected nodes

3 GraphSAGE layers + 1 output layer

Activation: ReLUa

Batch Normalization after each layer

Dropout: 0.3

Learns context-aware embeddings and captures fraud patterns via connected transactions

GAT Model - Graph Attention Network

3 Graph Attention Layers

Activation: ELU

Dropout: 0.3

Batch Norm after first 2 layers

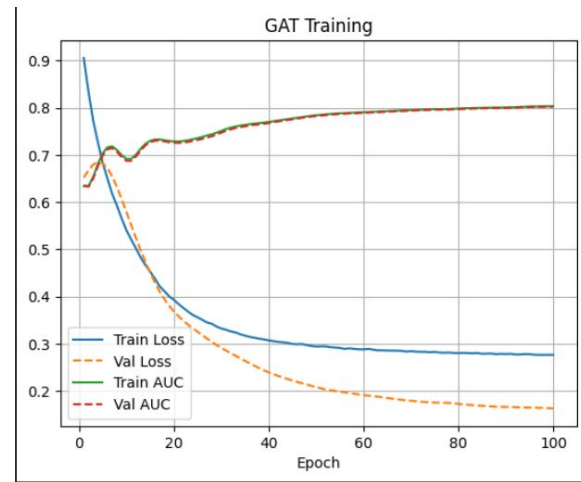
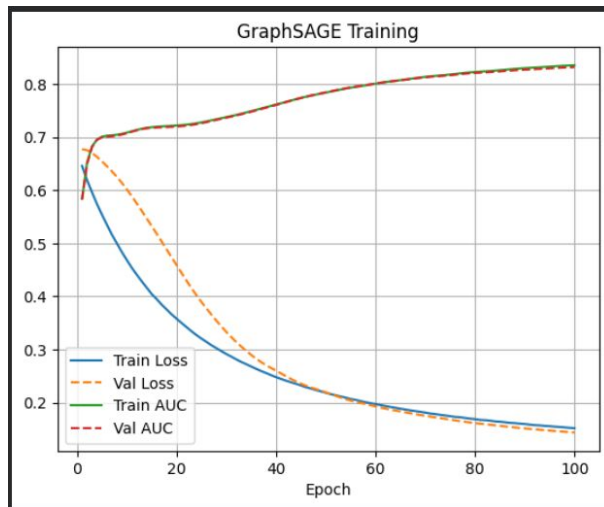
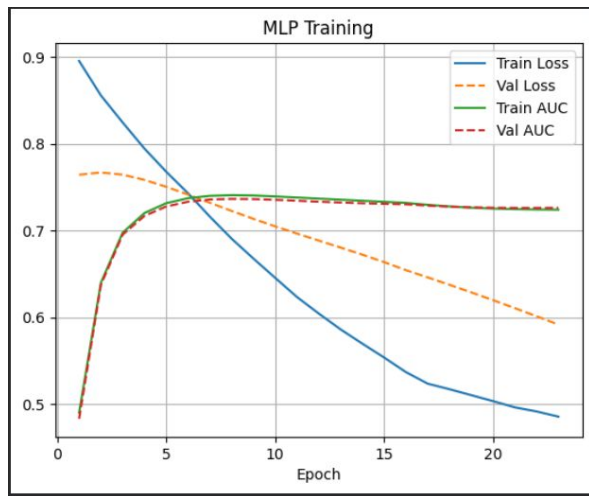
Uses edge_index + attention weights

Learns importance of neighbors

Not all neighbors are equal → model learns which ones matter more

In this , all the neighbours are not equal.

Training visualization

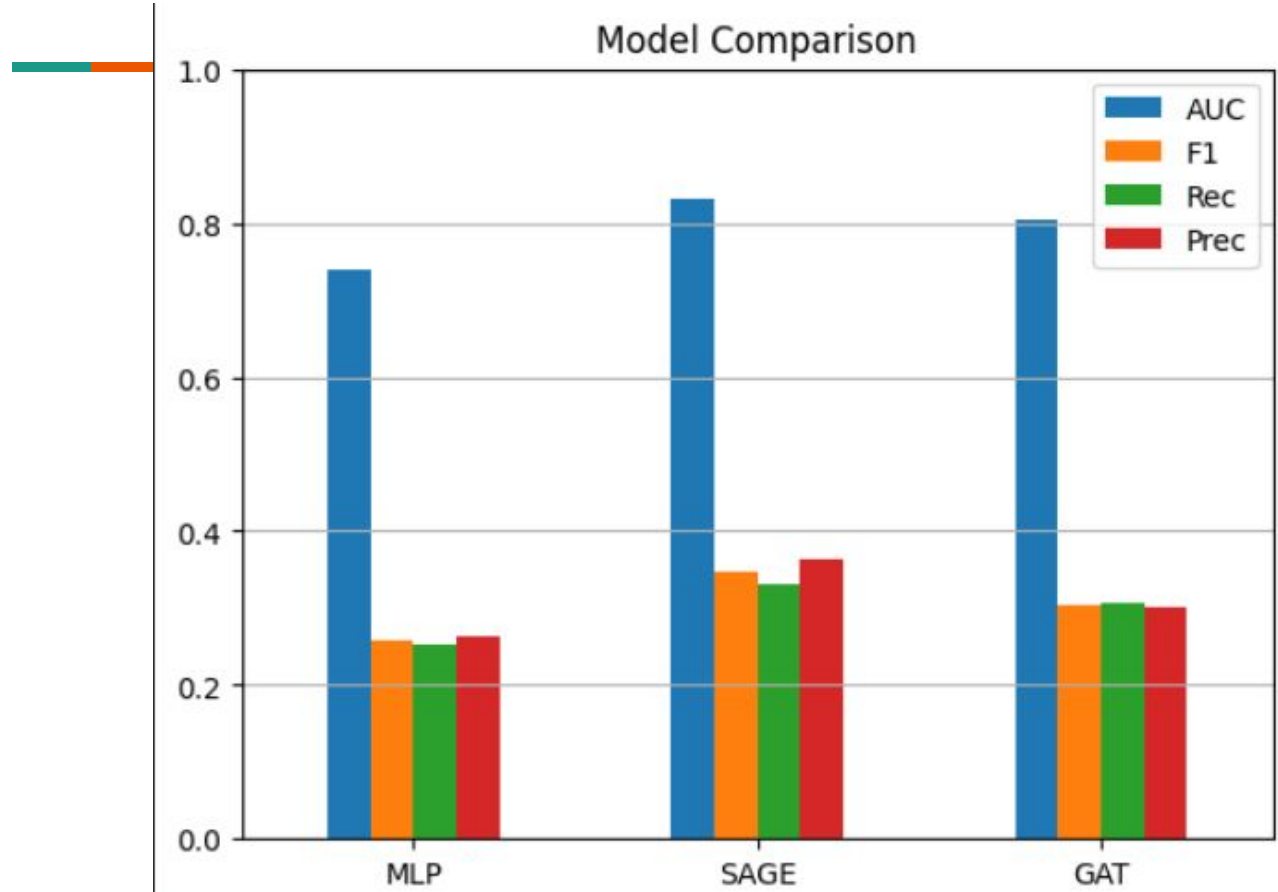




Results & Performance Metrics

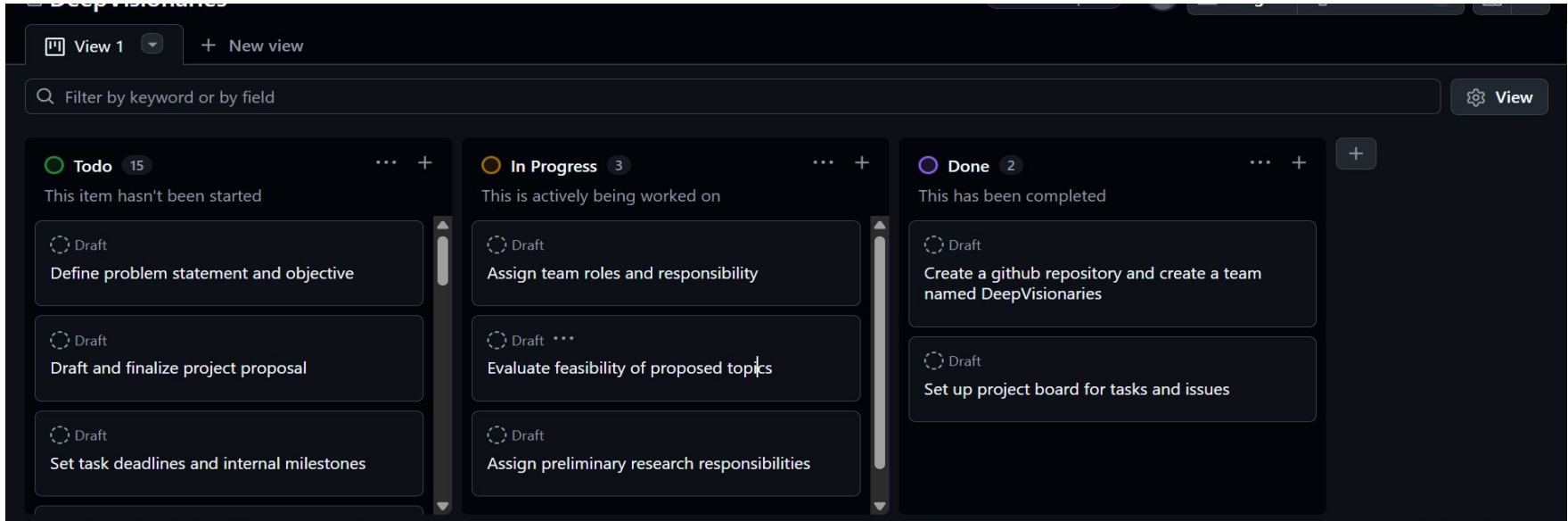
Metric (Fraud Class)	Baseline MLP	GraphSAGE GNN
AUC-ROC	0.7387	0.8313
F1-Score	0.2575	0.3475
Precision	0.2636	0.3213
Recall	0.2517	0.3169

Accuracy comparison of three models



Project Management Tool

INITIAL PLANNING FOR THE PROJECT



TARGET ACHIEVED TILL CHECKPOINT

The screenshot displays a GitHub Projects board for the repository 'ub-cse676-a-s26 / Projects / DeepVisionaries'. The board is organized into three columns: 'Todo' (3 items), 'In Progress' (3 items), and 'Done' (14 items). Each item is a card with a 'Draft' status and a description of the task. The 'Done' column also includes a 'final-project-deepvisionaries #2' item. The interface includes a search bar, a 'Filter by keyword or by field' dropdown, and a 'View' button. The top navigation bar shows the repository name, a search bar, and various icons for repository management.

ub-cse676-a-s26 / Projects / DeepVisionaries

Search: Type / to search

DeepVisionaries

Add status update

Insights

Workflows 6

View 1 + New view

Filter by keyword or by field

View

Todo 3

This item hasn't been started

unsuitable

Draft

Build demo interface for visualizing the outcomes

Draft

Run model comparisons and error analysis

+ Add item

In Progress 3

This is actively being worked on

Draft

Design a pre processing pipeline

Draft

Fine tune the model and check the outcomes

final-project-deepvisionaries #2

Track weekly project on github project board

Done 14

This has been completed

Draft

Create a github repository and create a team named DeepVisionaries

Draft

Set up project board for tasks and issues

Draft

Assign team roles and responsibility

PLANNED WORK FOR FINAL SUBMISSION

The screenshot displays a Trello project board for 'DeepVisionaries'. The board is organized into three columns: 'Todo' (5 items), 'In Progress' (4 items), and 'Done' (14 items). Each column contains a list of tasks, many of which are marked as 'Draft'.

Navigation and Search:

- Top bar: Search bar with placeholder 'Type / to search', icons for board management, and a user profile icon.
- Board header: 'DeepVisionaries' title, 'Add status update' button, and tabs for 'Insights', 'Workflows' (6), and a settings menu.
- View controls: 'View 1' dropdown and '+ New view' button.
- Filter bar: 'Filter by keyword or by field' search bar and a 'View' settings button.

Task Details by Column:

- Todo (5 items):**
 - Draft: Build demo interface for visualizing the outcomes
 - Draft: Run model comparisons and error analysis
 - Draft: Optimize the existing model
- In Progress (4 items):**
 - Draft: Design a pre processing pipeline
 - Draft: Fine tune the model and check the outcomes
 - final-project-deepvisionaries #2: Track weekly project on github project board
- Done (14 items):**
 - Draft: Create a github repository and create a team named DeepVisionaries
 - Draft: Set up project board for tasks and issues
 - Draft: Assign team roles and responsibility